

Week 6 - Class 12: Functions: Scope, Lifetime, Pass by Reference vs Value

Why This Matters

When writing functions you must understand:

- Scope: where a variable is visible.
- Lifetime: how long it exists in memory.
- Pass by Value: function gets a copy.
- Pass by Reference: function uses original.

Without this, you risk bugs, wasted memory, and accidental modifications.

Scope: Where a Variable Exists

- Local: inside a function or block, visible only there.
- Global: declared outside all functions, visible everywhere.
- Block: inside if/for/while, visible only in that block.

```
int globalVar = 10; // global

int main(){
    int localVar = 5;
    if(true){
        int blockVar = 3;
        cout<<blockVar;
    }
    // cout<<blockVar; // error, out of scope
}
```

Lifetime: How Long Variables Exist

- Local variables: created when block starts, destroyed when block ends.
- Global variables: created at program start, destroyed at program end.
- Static variables: created once, live for the whole program (explained below).

Pass by Value

Function gets a copy of variable. Caller not affected.

```
void addOne(int x){
    x = x + 1;
}

int main(){
    int a = 5;
    addOne(a);
    cout<<a; // 5 unchanged
}
```

Pass by Reference

Function works with original variable. Changes affect caller.

```
void addOne(int& x){
    x = x + 1;
}

int main(){
    int a = 5;
    addOne(a);
    cout<<a; // 6
}
```

Const References

const references allow read-only access without copying.

```
void printValue(const string& s){
    cout<<s<<"\n";
}
```

ASCII Diagram: Value vs Reference

Pass by Value: a: [5], copy: [5]

Pass by Reference: a: [5], ref alias -> same box

Static Variables: Making Data Last

Normally, local variables are created when function starts and destroyed when it ends.

Static variables are created once, kept for the whole program, and reused every call.

They are still private to that function.

Normal vs Static Example

```
void demo(){
    int normal = 0;
    static int persistent = 0;
    normal++;
    persistent++;
    cout << "normal="<<normal<<"", persistent="<<persistent<<"\n";
}

int main(){
    demo(); // normal=1,persistent=1
    demo(); // normal=1,persistent=2
    demo(); // normal=1,persistent=3
}
```

Why Static is Useful

1. Prevents waste: not recreated every call.
2. Keeps data private: unlike globals, other functions can't see it.
3. Tracks state: remembers values between calls.

Static Const Variables

static + const creates a permanent constant used inside the function.

- const alone: recreated every call (new memory each time).

- static const: created once, never changes, reused every call.

Example:

```
void printPi(){
    static const double PI = 3.14159;
    cout<<PI<<"\n";
}
```

Static vs Global

- Global const: visible everywhere.

- Static const: visible only in that function, but efficient (one copy).

Static variables provide efficiency and encapsulation.

Analogy

- Normal variable: disposable cup, new each call.

- Static variable: reusable mug, same each time.

- Static const: mug labeled permanently, one copy, never changes.

C++14 Fundamentals - Student Edition

Common Mistakes

1. Forgetting & when intending reference.
2. Passing large objects by value instead of const reference.
3. Forgetting const when reference shouldn't modify.
4. Overusing globals instead of static.

Exercises

1. Write `doublelt(int x)` pass by value. Show caller unchanged.
2. Rewrite as `doublelt(int& x)`. Show caller changes.
3. Write `printMessage(const string& msg)`.
4. Create `counter()` with static variable, call multiple times.
5. Experiment with global variable modified across functions.

Project Tie-In

Expand Function Library:

- Add pass-by-value and pass-by-reference functions.
- Add `counter()` using static variable.
- Discuss when to use value vs reference vs const reference.

This prepares you for efficient and safe functions in larger projects.